

Heaven's Light is Our Guide

RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY



A PROJECT REPORT

ON

**Conveyor Belt with Automated Storage and Retrieval
System (ASRS)**

Prepared by

Punno Chandra Saha (2005002)

Md. Rafiul Islam (2005004)

Md. Ferdaous Al- Farabe (2005023)

Habibur Rahman Alamin (2005025)

Ifte kharul Islam Nahid (2005034)

Rupok Islam Avi (2005058)

Department of Industrial & Production Engineering

16 September, 2025

CANDIDATES' DECLARATION

It is hereby declared that this thesis or any part of it has not been submitted elsewhere for the award of any degree or diploma.

.....
Punno Chandra Saha

.....
Md. Rafiul Islam

.....
Md. Ferdaous Al- Farabe

.....
Habibur Rahman Alamin

.....
Ifte kharul Islam Nahid

.....
Rupok Islam Avi

ABSTRACT

This project presents an Automated Storage and Retrieval System (ASRS) integrated with a conveyor belt and color-based sorting mechanism to demonstrate a compact and cost-effective automation solution. The system is controlled by an ESP32 microcontroller and utilizes sensors such as IR, ultrasonic, and a TCS725 color sensor for object detection, classification, and feedback control. Based on detected parameters, items are automatically transported and sorted into predefined storage bins, minimizing human intervention. Key components include servo motors for precise handling, and DC and gear motors for conveyor operation, all built with affordable DIY materials such as wood frames, 3D-printed parts, and modular wiring to ensure accessibility and flexibility. This ASRS model improves efficiency, reduces labor dependency, and is suitable for small-scale warehousing, packaging lines, recycling facilities, and research laboratories. Additionally, it serves as a practical learning platform in the fields of automation, robotics, and mechatronics.

Keywords: ASRS, ESP32, Conveyor, Color Sorting, Sensors, Automation.

ACKNOWLEDGEMENT

First and foremost, the authors express their gratitude to almighty Allah for providing us with the power and ability to complete the project report.

The authors would like to thank their respected supervisor **Sonia Akhter**, Associate Professor, Department of Industrial & Production Engineering, Rajshahi University of Engineering & Technology, Rajshahi-6204, Bangladesh, and **Md. Limonur Rahman Lingkon**, Lecturer, Department of Industrial & Production Engineering, Rajshahi University of Engineering & Technology, Rajshahi-6204, Bangladesh for their guidance, assistance, encouragement, and sympathetic cooperation throughout this project. It would have been difficult to complete this case project without their encouragement and close monitoring.

The authors would also like to thank all of the esteemed teachers in the Department of Industrial and Production Engineering for their well-wishes.

Authors

Punno Chandra Saha

Md. Rafiul Islam

Md. Ferdaous Al- Farabe

Habibur Rahman Alamin

Ifte kharul Islam Nahid

Rupok Islam Avi

CONTENTS

CANDIDATES' DECLARATION	I
ABSTRACT	II
ACKNOWLEDGEMENT	III
LIST OF TABLES	V
LISTS OF FIGURES	VI
CHAPTER	Page No.
CHAPTER 1: INTRODUCTION	1-2
CHAPTER 2: LITERATURE REVIEW	3-4
CHAPTER 3: SYSTEM COMPONENTS	5-15
CHAPTER 4: WORKING PRINCIPLECIRCUIT DESIGN	16-20
CHAPTER 5: COST ANALYSIS	21-22
CHAPTER 6: FINAL PROTOTYPE	23
CHAPTER 6: CONCLUSION	24
REFERENCES	25
APPENDIX	26-33

LIST OF TABLES

Table No.	Title	Page No.
Table 1:	List of Components.....	5
Table 5.1:	Cost of Components.....	22

LIST OF FIGURES

Figure No.	Title	Page No.
Fig 3.1.	ESP32 Microcontroller.....	06
Fig 3.2.	Gear Motor	07
Fig 3.3.	Servo Motor MG996.....	07
Fig 3.4.	DC motor	08
Fig 3.5.	Color Sensor TCS725	08
Fig 3.6.	IR Sensor	09
Fig 3.7:	Conveyer Belt	09
Fig 3.8.	Power Cable	10
Fig 3.9:	12V 2A PSU	10
Fig 3.10.	Buck Converter	11
Fig 3.11.	PCB	13
Fig 3.12:	Male-Female Rail.....	13
Fig 3.13.	Relay 5V.....	14
Fig 3.14.	D882 Transistor	15
Fig 3.15.	Buzzer	15
Fig 4.1.	Workflow of ASRS with conveyer belt system.	16
Fig 4.2.	Circuit diagram of ASRS with conveyer belt system	20
Fig 4.3	Pcb design of ASRS with conveyer belt system	20
Fig 6.1.	Different views of final prototypes	23

CHAPTER 1

INTRODUCTION

1.1 Introduction

Automation and intelligent control systems are transforming traditional industrial processes by enhancing speed, precision, and operational reliability. Among such innovations, the Automated Storage and Retrieval System (ASRS) has emerged as a vital component in modern logistics and manufacturing environments. These systems are designed to automatically store and retrieve products or components as needed, minimizing human intervention and increasing throughput.

In this project, we have developed a miniaturized ASRS model integrated with a conveyor belt system, and color detection. The entire system is controlled using a microcontroller (Arduino Uno) and is built using low-cost, DIY-friendly components such as DC motors, servo motors, sensors, and basic mechanical hardware.

The conveyor system transports objects to bin, when the bin is full with products automatically based on color classification detected by a color sensor (TCS3200). Sensors such as IR and ultrasonic modules provide position feedback and aid in automated decision-making.

This integrated project is not only a representation of how multiple measurement and control elements can work together in an automated environment but also serves as a practical platform for learning and experimentation in the fields of instrumentation and mechatronics

1.2 Problem Statement

In many small-scale industrial setups and educational laboratories, the handling, sorting, and storage of items are still performed manually. These traditional methods often lead to increased labor costs, reduced accuracy, and inefficient use of time and space. Moreover,

such systems lack the intelligence to detect and respond to mechanical faults or operational irregularities, which can result in downtime, component damage, and safety hazards.

There exists a significant need for a compact, affordable, and sensor-driven automated system that can perform item transportation, sorting, and storage. This would not only reduce human effort but also improve the efficiency and safety of the entire process.

1.3 Objectives

1. Design and develop a compact and affordable ASRS prototype integrated with a conveyor belt system.
2. Implement color-based object sorting using sensors (TCS3200, IR, and ultrasonic) for automated classification and placement.
3. Provide a practical platform for learning and experimentation in automation, instrumentation, and mechatronics.

1.4 Application Areas

1. **Small-Scale Warehousing:** Automate inventory management for small businesses or workshops.
2. **Packaging Lines:** Sort and transport lightweight items in low-budget packaging setups.
3. **Recycling Facilities:** Separate materials (e.g., plastics, metals) based on color or type.
4. **Research Prototypes:** Serve as a foundation for testing advanced algorithms (e.g., machine learning for object recognition).

CHAPTER 2

LITERATURE REVIEW

Automated Storage and Retrieval Systems (ASRS) are widely adopted in modern manufacturing, warehousing, and logistics due to their ability to improve efficiency, reduce labor dependency, and increase accuracy. The integration of embedded control systems, sensor technologies, and fault detection methods has been a major area of research, especially for small-scale educational and industrial applications.

Choy et al. [1] highlighted the transformative impact of ASRS on warehouse management by demonstrating how automation can significantly increase accuracy, reduce operating costs, and optimize space utilization. Their study underlines the importance of integrating real-time control and monitoring systems in automated logistics.

Kumar and Balamurugan [2] developed an Arduino-based ASRS model with conveyor and robotic components for small-scale automation. Their prototype showed that such systems are feasible for academic and experimental use, especially when budget constraints prevent the use of industrial-grade equipment.

The use of color sensors for real-time sorting has also been extensively studied. In particular, Khan and Ali [3] created a color sorting system using the TCS3200 sensor and Arduino, successfully distinguishing objects based on RGB values under different lighting conditions. A similar approach by Singh et al. [4] focused on using these sensors in conveyor-based automation for the packaging industry.

Robotic arms with servo-based control systems have been implemented in pick-and-place tasks. Dey and Roy [5] constructed a robotic arm using SG90 and MG996R servo motors controlled by an Arduino Uno. Their study validated the arm's ability to sort and position lightweight objects based on sensor input.

For accurate motion and safety, fault detection plays a critical role. Kumar et al. [6] proposed a real-time embedded fault detection system using current sensors (ACS712),

limit switches, and encoders. Their results showed improved reliability and prevention of hardware failure in automated operations.

Integration of IR and ultrasonic sensors for object detection and feedback is common in automation. Sharma and Das [7] implemented these sensors for line-following and obstacle detection in an autonomous mobile robot, similar to the AGV function in ASRS systems.

Arduino-based automation has gained popularity due to its simplicity, affordability, and versatility. Singh and Srivastava [8] successfully developed a complete warehouse model using Arduino Uno, conveyor motors, and sensors. Their work highlighted the benefits of using microcontrollers for synchronizing mechanical and electrical components in automation systems.

LCD and visual display modules offer valuable feedback in real-time systems. Sinha et al. [9] demonstrated how 16x2 LCD displays, when used with Arduino, can improve system interaction by showing status, fault messages, and sensor outputs, thereby enhancing usability and debugging.

Lastly, continuous advancements in low-cost automation have been supported by the use of modular DIY components. Ahmed and Malik [10] showcased how embedded systems could be combined with mechanical structures to create flexible, scalable automation setups suitable for academic environments.

These studies collectively inform the development of this project by providing insights into cost-effective design, sensor integration, embedded control, and fault detection in ASRS and conveyor-based automation.

CHAPTER 3

SYSTEM COMPONENTS

This chapter describes the components of the project and their functions.

3.1 Components

The list of the components and their functions are described below:

Table 1: List of Components

SL	Component	Qty
1	Eps32 microcontroller	1
2	Gear motor	1
3	Servo motor mg996	2
4	DC motor	3
5	Color sensor tcs725	1
6	IR sensor	2
7	Belt	1
8	Power cable	1
9	12V 2A PSU	1
10	5V 2A PSU	1
11	Buck converter	1
12	Roller	2
13	Bearing	4
14	3D printed parts	1
15	Wood frame	1
16	Wire	1
17	PCB	1
18	Male-Female rail	4
19	PVC board	1
20	Relay 5V	1
21	D882 transistor	3
22	Buzzer	1
23	Others	1

3.2 Specification of Components

1. ESP32 Microcontroller

The ESP32 is a compact, dual-core microcontroller with integrated Wi-Fi and Bluetooth connectivity, designed for advanced automation. In the ASRS, it serves as the central control hub, executing programmed logic, processing sensor feedback, and managing communication between modules. Its high-speed processing ensures accurate conveyor coordination, while low-power efficiency supports continuous operation with reliable fault detection and responsive decision-making.



Fig 3.1. ESP32 Microcontroller

2. Gear Motor

A gear motor combines a motor with a gearbox to deliver high torque at low speeds. In the ASRS, it powers conveyor movement, ensuring steady and controlled transportation of stored items. The gearbox enhances load-handling capability while reducing motor strain. This provides smooth acceleration, precise stopping, and dependable mechanical performance for consistent retrieval operations without slippage or overloading.



Fig 3.2. Gear Motor

3. Servo Motor MG996

The MG996 servo motor offers high torque and angular precision, ideal for positioning mechanisms in storage systems. In this project, it controls directional movements, such as lifting, shifting, or locking items within the conveyor framework. Its robust metal gears withstand continuous duty cycles, while the precise pulse-width modulation ensures accurate positioning essential for error-free storage and retrieval automation.



Fig 3.3. Servo Motor MG996

4. DC Motor

A DC motor delivers direct rotational power from electrical energy, known for simplicity and efficiency. In the ASRS, it drives auxiliary mechanisms such as rollers or lifts,

providing continuous rotation where precise speed control is unnecessary. Its responsiveness supports smooth transitions in material handling, while its rugged design allows repetitive duty cycles under varying load conditions without excessive wear.



Fig 3.4. DC Motor

5. Color Sensor TCS725

The TCS725 color sensor detects RGB light intensity with high sensitivity and accuracy. Within the ASRS, it identifies item codes, packaging colors, or markers used in fault detection. Its photodiode array converts reflected light into digital signals, enabling automated recognition and classification. This capability enhances sorting precision and ensures proper tracking of items through the conveyor belt system.

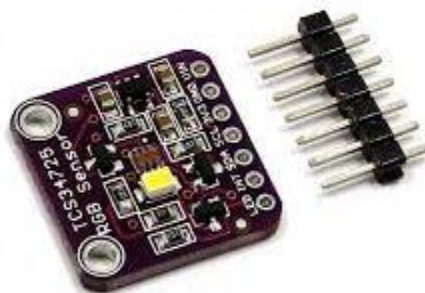


Fig 3.5. Color Sensor TCS725

6. IR Sensor

An infrared (IR) sensor detects the presence or absence of objects by emitting and receiving infrared light. In the ASRS, it enables collision avoidance, position tracking, and proximity detection along the conveyor. It acts as a safeguard, halting operations when obstructions occur. Its rapid response and contactless detection contribute to reliable fault prevention and efficient system automation.



Fig 3.6. IR Sensor

7. Conveyor Belt

The conveyor belt is a flexible, durable looped medium for transporting items within the ASRS. Driven by rollers and motors, it enables smooth and consistent movement of loads. Its frictional surface ensures items remain stable while traveling. Constructed from resilient material, it resists wear during repetitive cycles, ensuring reliable, continuous operation essential for automated storage and retrieval systems.



Fig 3.7. Conveyor Belt

8. Power Cable

A power cable is a multi-conductor insulated wire designed to safely transmit electrical energy to devices. In the ASRS, it distributes regulated power from power supply units to motors, sensors, and control boards. Its protective insulation prevents short circuits and energy losses. Reliable cabling guarantees stable operation, protecting sensitive electronics while ensuring uninterrupted functionality of automation components.



Fig 3.8. Power Cable

9. 12V 2A PSU

The 12V 2A power supply unit converts AC mains to a stable 12V DC output with a current capacity of 2A. In the ASRS, it powers motors, relays, and mechanical drives requiring higher voltage. Its regulated output prevents surges and overheating, ensuring dependable energy delivery for consistent conveyor belt motion and mechanical stability in demanding operational cycles.



Fig 3.9. 12V 2A PSU

10. 5V 2A PSU

The 5V 2A power supply unit provides regulated DC output for microcontrollers and low-power devices. In the ASRS, it energizes the ESP32, sensors, and signal-processing circuits. Its precise voltage regulation protects sensitive electronics from fluctuations. With sufficient current delivery, it supports stable logic operations and fault detection, ensuring uninterrupted performance of control and monitoring subsystems.

11. Buck Converter

A buck converter is a DC-to-DC step-down regulator that efficiently reduces voltage levels. In the ASRS, it ensures safe power distribution by adapting 12V input into usable levels for sensors or logic devices. Its high efficiency minimizes energy loss and heat generation, while precise control maintains stable voltages. This guarantees reliable integration of multiple components with diverse power needs.



Fig 3.10. Buck Converter

12. Roller

Rollers are cylindrical components mounted along the conveyor to support and guide belt movement. In the ASRS, they provide low-friction rotation, reducing energy consumption while maintaining belt tension. Constructed from durable materials, they withstand repetitive load-bearing cycles. By ensuring smooth and even transport, rollers enable efficient material handling, prevent belt misalignment, and enhance overall conveyor reliability.

13. Bearing

Bearings are precision-engineered mechanical elements that reduce friction between moving parts. In the ASRS, they support motor shafts, rollers, and other rotating components. By minimizing resistance, bearings extend operational life and enhance system efficiency. Their robust construction withstands repeated loads, ensuring consistent rotation. This reliability is critical for smooth conveyor performance and precise storage-and-retrieval functionality.

15. Wood Frame

The wood frame acts as the structural base of the ASRS, providing rigidity and alignment for all mounted components. It is precisely cut to house motors, sensors, and conveyor assemblies. Its stable construction ensures proper distribution of loads, reducing vibration and mechanical stress. The frame's simplicity supports modular assembly while maintaining strength necessary for reliable automated operations.

16. Wire

Wires are conductive metallic strands insulated for safety, enabling electrical connectivity across the ASRS. They interlink sensors, motors, and controllers with power supplies and signal pathways. High-quality wiring ensures minimal resistance, preventing energy loss and overheating. Their flexibility supports organized routing, while robust insulation prevents electrical hazards. Proper wiring guarantees seamless communication and consistent power distribution.

17. PCB

A printed circuit board (PCB) is a flat substrate etched with conductive pathways for component interconnection. In the ASRS, it consolidates control circuits, power regulation, and sensor inputs into a compact design. Its organized layout reduces wiring complexity, improves reliability, and enhances maintainability. The PCB enables efficient signal processing, supporting precise coordination of conveyor automation and fault detection.



Fig 3.11. PCB

18. Male-Female Rail

The male-female rail connector provides modular electrical connectivity between detachable sections of the ASRS. It ensures secure, polarized mating, preventing misalignment during assembly. These rails simplify component replacement and enable quick disconnection for maintenance. By ensuring stable current flow and mechanical rigidity, they support reliable communication between subsystems, vital for efficient operation of automated storage and retrieval.



Fig 3.12. Male-Female Rail

19. PVC Board

A PVC board is a lightweight, durable plastic sheet used for mounting and insulation. In the ASRS, it provides a base for electrical circuits, housing sensors or protective covers. Its resistance to moisture, heat, and wear ensures long-term stability. Easy machinability allows precise customization, while its insulating properties safeguard sensitive electronics from short circuits and environmental hazards.

20. Relay 5V

The 5V relay is an electromechanical switch that controls high-power loads using low-power signals. In the ASRS, it enables the microcontroller to operate motors or alarms indirectly. Its coil responds to control signals, triggering internal contacts that switch external circuits. This isolation ensures safe operation, allowing low-voltage logic to manage high-current components without direct electrical stress.



Fig 3.13. Relay 5V

21. D882 Transistor

The D882 transistor is a general-purpose NPN semiconductor used for amplification and switching. In the ASRS, it drives relay coils, motors, or buzzers by handling higher currents from control signals. Its fast response and thermal durability ensure stable

performance under repetitive switching cycles. By acting as an efficient electronic switch, it contributes to reliable fault signaling and actuation.



Fig 3.14. D882 Transistor

22. Buzzer

The buzzer is an electromechanical sound-emitting device used for audible alerts. In the ASRS, it signals system faults, successful operations, or emergency halts with clear tones. Its compact design produces reliable acoustic output when triggered by control signals. Providing immediate human feedback, the buzzer enhances safety, ensures timely attention, and supports effective monitoring of automated storage operations.



Fig 3.15. Buzzer

CHAPTER 4

WORKING PRINCIPLE & CIRCUIT DESIGN

This section details the technical implementation and operational logic of the ASRS and integrated conveyor belt system. It begins by illustrating the system's workflow, which outlines the step-by-step sequence of operations from storage to retrieval. Following this, the circuit diagram is presented, providing a complete electronic schematic of how the microcontroller, sensors, motors, and other components are interconnected. Finally, the PCB design is shown, translating the abstract circuit diagram into the physical, compact layout of the custom-printed circuit board that forms the system's hardware core.

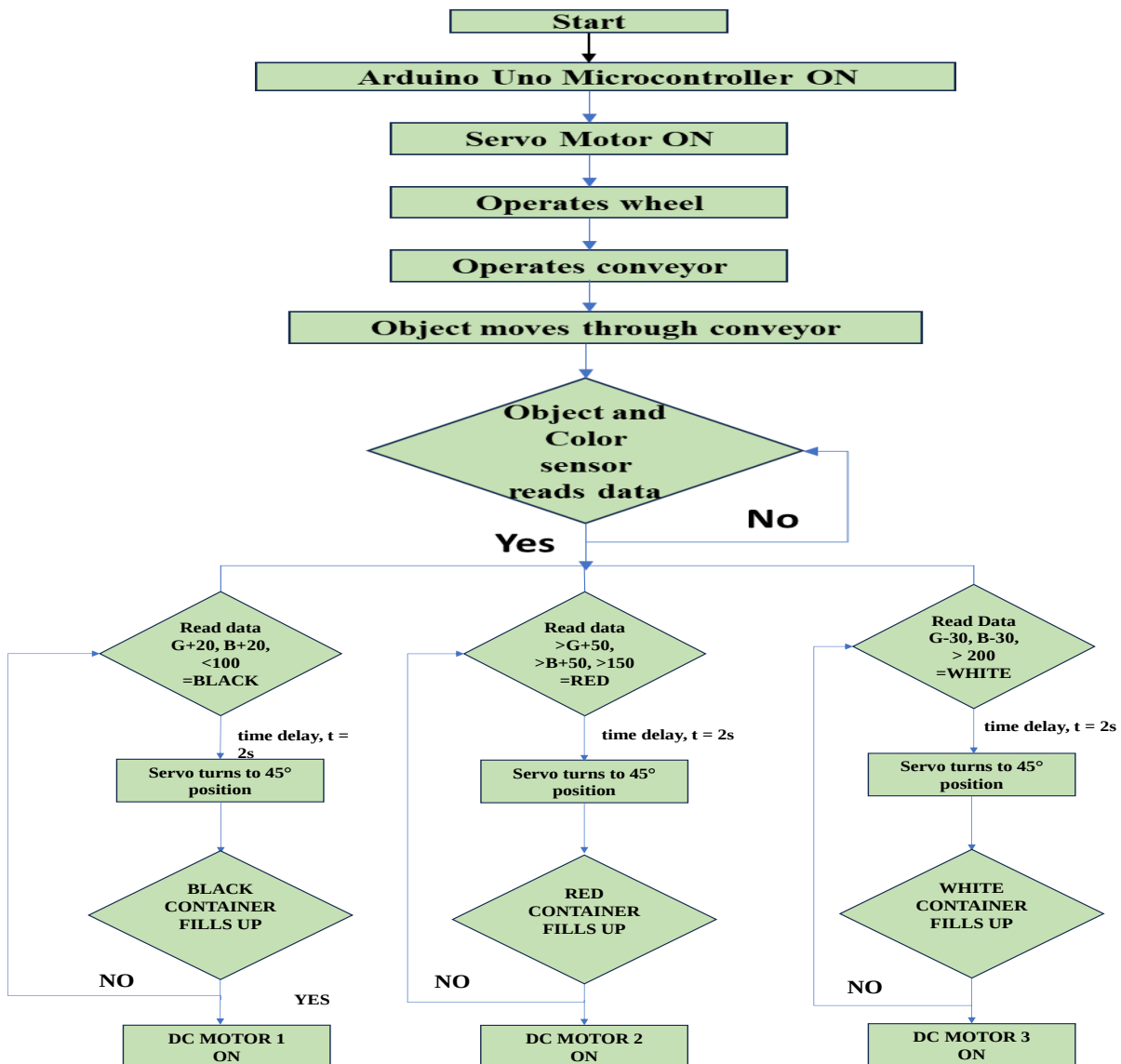


Fig 4.1. Workflow of ASRS with conveyer belt system

The steps for the workflow are given below:

Phase 1: System Initialization

1. Step 1: Power On

- The process begins when the system is powered on.

2. Step 2: Microcontroller Boot

- The Arduino Uno microcontroller initializes. This is the "brain" of the entire operation and will control all other components.

3. Step 3: Servo Motor Activation

- The Arduino powers on the Servo Motor. This motor is responsible for steering objects off the main conveyor belt into the correct chutes or containers.

Phase 2: Object Transportation

4. Step 4: Conveyor Belt Operation

- The main conveyor belt is activated (likely by a separate motor controlled by the Arduino). It begins moving continuously.

5. Step 5: Object Movement

- An object is placed on the moving conveyor belt, which transports it toward the sensing and sorting station.

Phase 3: Object Detection & Identification

6. Step 6: Data Reading

- A color sensor (e.g., TCS3200) reads the object's color as it passes by.
- The sensor sends data (Red, Green, Blue values and overall luminosity) to the Arduino.

7. Step 7: Color Classification

- The Arduino compares the sensor data against predefined thresholds to classify the object:
- **Condition:** $G+20, B+20, <100$
 - **Result:** BLACK -> The object is identified as black.

- **Condition:** >G+50, >B+50, >150
 - **Result:** RED -> The object is identified as red.
- **Condition:** G-30, B-30, >200
 - **Result:** WHITE -> The object is identified as white.

Phase 4: Sorting & Storage

8. Step 8: Servo Motor Positioning

- Based on the identified color, the Arduino commands the Servo Motor to turn to a specific position (e.g., 45°). This position aligns a guide or a secondary chute with the path to the correct container.

9. Step 9: Object Diversion

- The object continues moving on the conveyor until it reaches the servo-guided diverter, which pushes it or allows it to roll into the correct chute, leading to its designated container.

10. Step 10: Container Filling

- The object drops into its specific container (e.g., Black Container, Red Container, White Container). The system notes that the correct "storage" location is being filled.

Phase 5: Post-Sorting Action (Optional)

11. Step 11: DC Motor Activation

- As a final step (which could be triggered by the object entering the chute or by a timer), the Arduino turns ON a specific DC Motor associated with that container.
- DC Motor 1 ON for the Black container.
- DC Motor 2 ON for the Red container.
- DC Motor 3 ON for the White container.
- This motor could serve various purposes, such as:

- Powering a smaller belt inside the container to organize stored items.
- Compacting the items in the container.
- Notifying an operator that a container is reaching capacity.

The process then loops back to **Step 5**, waiting for the next object to be placed on the conveyor.

Fig 4.2 illustrates the complete ASRS circuit diagram, which is orchestrated by a central microcontroller. This microcontroller manages the system's actuators, including two DC motors (Motor 1, Motor 2) controlled via a dual H-bridge motor driver (using pins D4-D9 for direction and speed) and a servo motor (using PWM pin D10 for positioning). For sensing, an RGB colour sensor is integrated to identify items, communicating with the microcontroller via the I2C protocol (A4/SDA and A5/SCL). The power system is split, with a 12V supply (VCC1) for the motors and a 9V supply (VCC2) for the driver's logic, while the microcontroller and sensor operate at 5V, all sharing a common ground.

Fig 4.3 presents the PCB design for the ASRS and conveyor belt system, which is based on an ESP32 microcontroller as the central processing unit. The board is designed to be powered by a +12V DC input and includes reverse polarity protection. It integrates all necessary peripherals, featuring dedicated headers for a servo motor, a color sensor (TCS3), two IR sensor modules, and an emergency stop button. For output, it utilizes an onboard relay (RELAY1) to control the high-power conveyor motor, along with connections for a buzzer. The layout is a single-sided design, evidenced by the use of several red jumper wires to route signals from the ESP32's I/O pins to the various components.

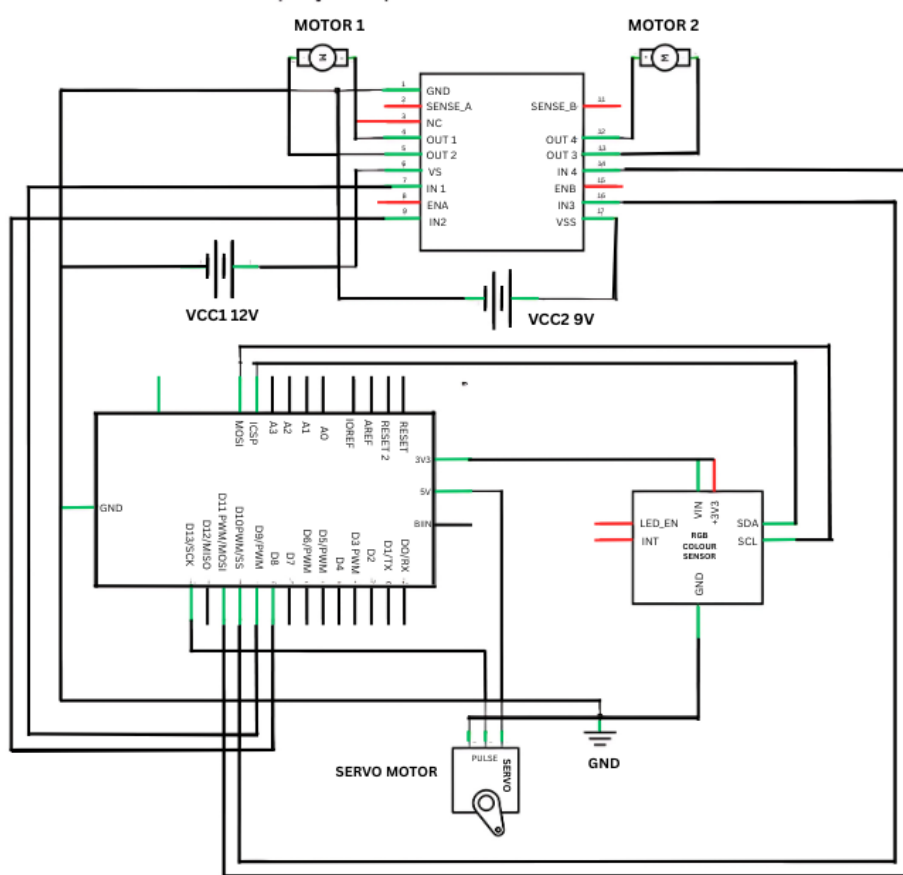


Fig 4.2. Circuit diagram of ASRS with conveyor belt system

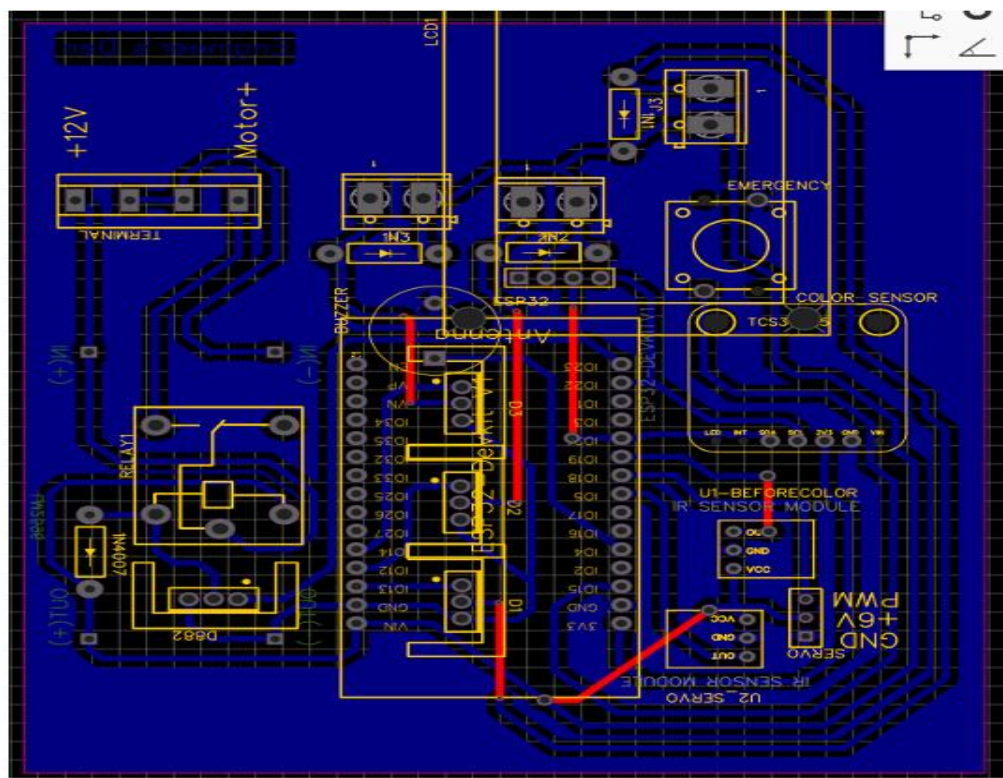


Fig 4.3. Pcb design of ASRS with conveyor belt system

CHAPTER 5

COST ANALYSIS

Cost analysis is the process of evaluating the costs associated with a project, product, or system to understand its financial requirements and efficiency. It involves identifying, categorizing, and quantifying all direct and indirect expenses, including materials, labor, equipment, and overheads. The primary purpose of cost analysis is to provide a clear understanding of the total investment needed, support budgeting decisions, and help in optimizing resource allocation.

In engineering and manufacturing projects, cost analysis plays a crucial role in assessing the feasibility and sustainability of a design. By systematically evaluating component costs, production methods, and operational expenses, engineers and project managers can make informed decisions to minimize costs while maintaining quality and functionality. It also provides a basis for comparing alternative solutions, negotiating procurement, and planning for future expansions or maintenance.

Overall, cost analysis ensures that financial resources are utilized efficiently, reduces the risk of unexpected expenditures, and enhances the overall economic viability of a project.

Table 5.1: Cost of Components

SL	Component	Qty	Unit Price (Tk)	Total (Tk)
1	Eps32 microcontroller	1	800	800
2	Gear motor	1	1000	1000
3	Servo motor mg996	2	600	1200
4	DC motor	3	80	240
5	Color sensor tcs725	1	600	600
6	IR sensor	2	150	300
7	Belt	1	300	300
8	Power cable	1	80	80
9	12V 2A PSU	1	550	550
10	5V 2A PSU	1	450	450
11	Buck converter	1	200	200
12	Roller	2	600	1200
13	Bearing	4	150	600
14	3D printed parts	1	350	350
15	Wood frame	1	800	800
16	Wire	1	200	200
17	PCB	1	600	600
18	Male-Female rail	4	30	120
19	PVC board	1	200	200
20	Relay 5V	1	60	60
21	D882 transistor	3	15	45
22	Buzzer	1	25	25
23	Others	1	300	300
	Total Cost			10,220

CHAPTER 6

FINAL PROTOTYPE

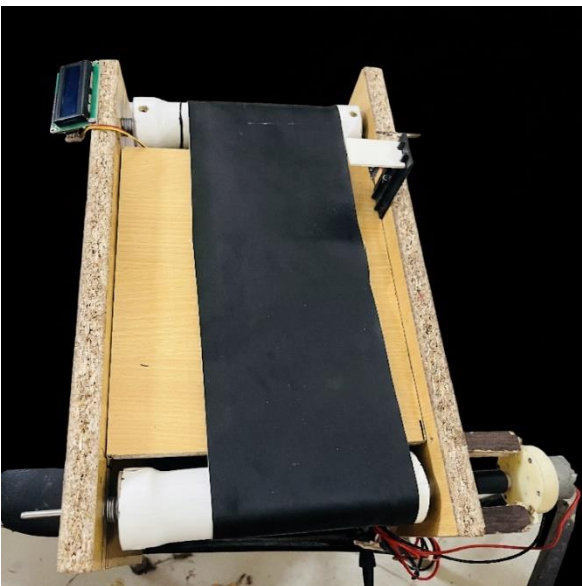
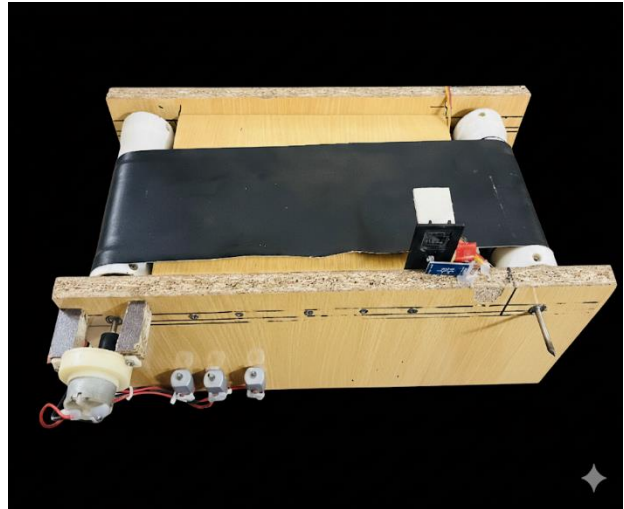
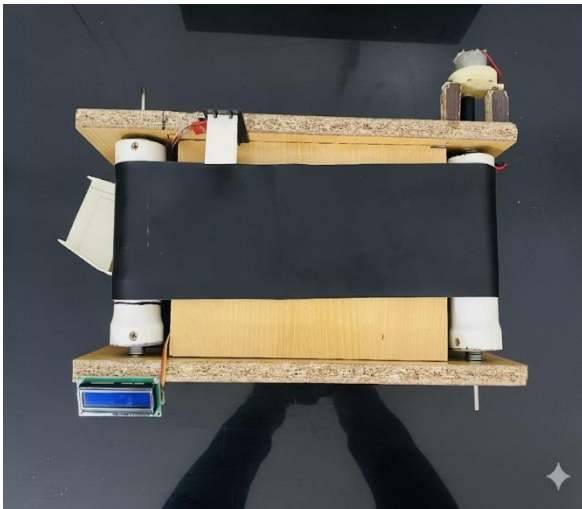


Fig 6.1. Different views of final prototypes

CHAPTER 7

CONCLUSION

This project successfully achieved its primary objectives by designing and developing a compact, cost-effective Automated Storage and Retrieval System (ASRS) integrated with a conveyor belt. The system addresses the inefficiencies of manual sorting in small-scale setups by providing an affordable automation solution, with a total component cost of 10,220 Taka.

The core objective of implementing automated, color-based sorting was successfully realized. The system uses an ESP32 microcontroller as its central control hub , which processes inputs from IR sensors and a TCS725 color sensor to detect and classify items. Based on this classification, servo motors are precisely controlled to sort objects into their designated storage bins, effectively minimizing the need for human intervention.

Constructed from accessible DIY materials, including a wood frame and 3D-printed parts , this prototype has proven to be an efficient solution that reduces labor dependency. Furthermore, it fulfills its goal of serving as a practical and valuable learning platform for students and researchers in the fields of automation, mechatronics, and robotics. This ASRS model is well-suited for various applications, including small-scale warehousing, packaging lines, and recycling facilities

REFERENCES

- [1] K. L. Choy, W. B. Lee, and V. Lee, "A real-time warehouse management system for perishable food," *Expert Systems with Applications*, vol. 36, no. 4, pp. 9026–9032, 2009.
- [2] S. Kumar and S. Balamurugan, "Design and Development of a Mini Automated Storage and Retrieval System (ASRS) using Arduino," *International Journal of Engineering Research and Applications (IJERA)*, vol. 10, no. 6, pp. 19–24, Jun. 2020.
- [3] R. Khan and M. Ali, "Arduino-Based Automatic Color Sorting Machine," *International Journal of Scientific and Research Publications (IJSRP)*, vol. 9, no. 4, pp. 95–99, Apr. 2019.
- [4] A. Singh, R. Ghosh, and A. Dutta, "Automated Object Sorting using TCS3200 Color Sensor with Arduino and Conveyor Belt," *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, vol. 8, no. 3, pp. 11–15, Mar. 2019.
- [5] S. Dey and S. Roy, "Design and Fabrication of a Pick and Place Robot with Arduino Microcontroller," *International Journal of Engineering Research and Technology (IJERT)*, vol. 7, no. 5, pp. 1–4, May 2018.
- [6] R. Kumar, S. Singh, and D. Sharma, "Low Cost Embedded Fault Detection System for Mechatronics Application," *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, vol. 5, no. 3, pp. 202–207, Mar. 2016.
- [7] A. Sharma and S. Das, "Obstacle Avoidance and Line Following Robot using IR and Ultrasonic Sensors," *International Journal of Science and Research (IJSR)*, vol. 6, no. 7, pp. 678–683, Jul. 2017.
- [8] R. Singh and S. Srivastava, "Arduino Based Smart Warehouse Automation System," *International Journal of Trend in Scientific Research and Development (IJTSRD)*, vol. 3, no. 4, pp. 1043–1046, Jun. 2019.
- [9] A. Sinha, P. Mishra, and M. Verma, "Real Time Status Display in Embedded Systems using LCD with I2C Module," *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, vol. 9, no. 5, pp. 4312–4316, May 2020.
- [10] A. Ahmed and T. Malik, "Development of Low-Cost Mechatronics-Based Conveyor Sorting System," *International Journal of Mechanical and Production Engineering Research and Development (IJMPERD)*, vol. 10, no. 4, pp. 217–224, Aug. 2020.

APPENDIX

Code:

```
#include <Wire.h>
#include <Adafruit_TCS34725.h>
#include <LiquidCrystal_I2C.h>

// Servo PWM configuration
#define SERVO_PIN 18
#define SERVO_CHANNEL 0
#define SERVO_FREQ 50
#define SERVO_RESOLUTION 16

// Pin definitions
#define BELT_RELAY_PIN 23
#define BOX_IR_PIN 5
#define OBJECT_IR_PIN 19
#define BUZZER_PIN 4
#define BLACK_MOTOR_PIN 13
#define WHITE_MOTOR_PIN 12
#define RED_MOTOR_PIN 14

// Servo positions
#define SERVO_RED 98
#define SERVO_BLACK 67
#define SERVO_WHITE 130

// Updated color thresholds based on your actual readings
// For RED: R is significantly higher than G and B
#define RED_MIN_R 200
#define RED_RG_DIFF 30
#define RED_RB_DIFF 30

// For WHITE: R, G, B are relatively balanced with high clear value
#define WHITE_MIN_C 450
```

```

#define WHITE_MAX_RG_DIFF 100
#define WHITE_MAX_RB_DIFF 150
#define WHITE_MAX_GB_DIFF 100

// For BLACK: R, G, B are high and very close to each other
#define BLACK_MIN_R 300
#define BLACK_MIN_G 250
#define BLACK_MIN_B 200
#define BLACK_MAX_RG_DIFF 60
#define BLACK_MAX_RB_DIFF 80
#define BLACK_MAX_GB_DIFF 60

// Number of color readings to take
#define NUM_READINGS 5
#define STABILIZATION_DELAY 800
#define READING_DELAY 200

// System states
enum SystemState { SCANNING, DETECTED_COLOR, BOX_FULL, BOX_EMPTY_WAIT
    };
SystemState currentState = SCANNING;
unsigned long stateTimer = 0;

// Initialize components
Adafruit_TCS34725 colorSensor =
    Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS, TCS34725_GAIN_4X);
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Box status
bool redBoxFull = false;
bool whiteBoxFull = false;
bool blackBoxFull = false;

// Detected color

```

```

enum DetectedColor { COLOR_RED, COLOR_WHITE, COLOR_BLACK,
    COLOR_UNDEFINED };
DetectedColor currentColor = COLOR_UNDEFINED;
DetectedColor lastDetectedColor = COLOR_UNDEFINED; // Track last detected color

// Color reading variables
unsigned long objectDetectedTime = 0;
bool waitingForStabilization = false;
int colorCounts[4]; // Array to count color detections: [RED, WHITE, BLACK, UNDEFINED]

void setup() {
    Serial.begin(115200);

    // Initialize pins
    pinMode(BELT_RELAY_PIN, OUTPUT);
    pinMode(BOX_IR_PIN, INPUT_PULLUP);
    pinMode(OBJECT_IR_PIN, INPUT_PULLUP);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(BLACK_MOTOR_PIN, OUTPUT);
    pinMode(WHITE_MOTOR_PIN, OUTPUT);
    pinMode(RED_MOTOR_PIN, OUTPUT);

    // Initialize motors to OFF
    digitalWrite(BLACK_MOTOR_PIN, LOW);
    digitalWrite(WHITE_MOTOR_PIN, LOW);
    digitalWrite(RED_MOTOR_PIN, LOW);

    // Configure LEDC for servo
    ledcSetup(SERVO_CHANNEL, SERVO_FREQ, SERVO_RESOLUTION);
    ledcAttachPin(SERVO_PIN, SERVO_CHANNEL);

    // Set initial servo position
    setServoAngle(SERVO_RED);
    Serial.println("Servo initialized at 98 degrees (RED position)");
}

```

```
delay(1000); // Give servo time to move
```

```
// Initialize LCD
```

```
lcd.init();
```

```
lcd.backlight();
```

```
lcd.print("Color Sorter");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("Initializing...");
```

```
// Initialize color sensor
```

```
if (!colorSensor.begin()) {
```

```
    Serial.println("Color sensor not found!");
```

```
    lcd.setCursor(0, 1);
```

```
    lcd.print("Sensor Error!");
```

```
    while (1); // Halt if sensor not found
```

```
}
```

```
// Start belt - NOTE: Relay is inverted, so LOW turns ON the belt
```

```
digitalWrite(BELT_RELAY_PIN, LOW);
```

```
Serial.println("Belt started (relay LOW)");
```

```
lcd.clear();
```

```
lcd.print("System Ready");
```

```
delay(1000);
```

```
lcd.clear();
```

```
}
```

```
void loop() {
```

```
    // Check for box full condition (can happen in any state)
```

```
    checkBoxFull();
```

```
    // Update LCD based on current state
```

```
    updateLCD();
```

```

// State machine
switch (currentState) {
    case SCANNING: scanForObjects(); break;
    case DETECTED_COLOR: processDetectedColor(); break;
    case BOX_FULL: handleFullBox(); break;
    case BOX_EMPTY_WAIT: handleBoxEmptyWait(); break;
}
}

// Function to set servo angle using direct PWM
void setServoAngle(int angle) {
    int pulseWidth = map(angle, 0, 180, 500, 2400);
    int dutyCycle = map(pulseWidth, 0, 20000, 0, 65535);
    ledcWrite(SERVO_CHANNEL, dutyCycle);
}

// Check if any box is full
void checkBoxFull() {
    if (digitalRead(BOX_IR_PIN) == LOW) {
        if (currentState != BOX_FULL && currentState != BOX_EMPTY_WAIT) {
            // Stop belt immediately - NOTE: Relay is inverted, so HIGH turns OFF the belt
            digitalWrite(BELT_RELAY_PIN, HIGH);
            Serial.println("Belt stopped (relay HIGH)");
            currentState = BOX_FULL;
            stateTimer = millis();

            // Determine which box is full based on last detected color
            if (lastDetectedColor == COLOR_RED) { redBoxFull = true;
            digitalWrite(RED_MOTOR_PIN, HIGH); Serial.println("RED box full - motor ON"); }
            else if (lastDetectedColor == COLOR_WHITE) { whiteBoxFull = true;
            digitalWrite(WHITE_MOTOR_PIN, HIGH); Serial.println("WHITE box full - motor
ON"); }

```

```

    else if (lastDetectedColor == COLOR_BLACK) { blackBoxFull = true;
digitalWrite(BLACK_MOTOR_PIN, HIGH); Serial.println("BLACK box full - motor
ON"); }
    else {
        if (currentColor == COLOR_RED) { redBoxFull = true;
digitalWrite(RED_MOTOR_PIN, HIGH); Serial.println("RED box full (current color) -
motor ON"); }
        else if (currentColor == COLOR_WHITE) { whiteBoxFull = true;
digitalWrite(WHITE_MOTOR_PIN, HIGH); Serial.println("WHITE box full (current
color) - motor ON"); }
        else if (currentColor == COLOR_BLACK) { blackBoxFull = true;
digitalWrite(BLACK_MOTOR_PIN, HIGH); Serial.println("BLACK box full (current
color) - motor ON"); }
        else { redBoxFull = true; digitalWrite(RED_MOTOR_PIN, HIGH);
Serial.println("RED box full (default) - motor ON"); }
    }

    // Long beep for box full
    beep(1000);
}
}
}

// Object just detected, start stabilization timer
void scanForObjects() {
    if (digitalRead(OBJECT_IR_PIN) == LOW) {
        if (!waitingForStabilization) {
            objectDetectedTime = millis();
            waitingForStabilization = true;
            Serial.println("Object detected, stabilizing...");
            for (int i = 0; i < 4; i++) { colorCounts[i] = 0; }
        }

        // Check if stabilization period is complete

```

```

if (millis() - objectDetectedTime >= STABILIZATION_DELAY) {
    Serial.println("Stabilization complete, taking readings...");
    for (int i = 0; i < NUM_READINGS; i++) {
        uint16_t r, g, b, c;
        colorSensor.getRawData(&r, &g, &b, &c);

        // Print values for debugging
        Serial.print("Reading "); Serial.print(i + 1); Serial.print(": R="); Serial.print(r);
        Serial.print(" G="); Serial.print(g); Serial.print(" B="); Serial.print(b); Serial.print("
C="); Serial.println(c);

        // Calculate ratios for better color distinction
        float rRatio = (float)r / (r + g + b);
        float gRatio = (float)g / (r + g + b);
        float bRatio = (float)b / (r + g + b);

        // Determine color for this reading
        DetectedColor readingColor;
        if (isBlack(r, g, b, rRatio, gRatio, bRatio)) { readingColor = COLOR_BLACK;
        Serial.println(" -> BLACK"); }
        else if (isWhite(r, g, b, c, rRatio, gRatio, bRatio)) { readingColor = COLOR_WHITE;
        Serial.println(" -> WHITE"); }
        else if (isRed(r, g, b, rRatio, gRatio, bRatio)) { readingColor = COLOR_RED;
        Serial.println(" -> RED"); }
        else { readingColor = COLOR_UNDEFINED; Serial.println(" -> UNDEFINED"); }

        // Increment count for this color
        colorCounts[readingColor]++;

        // Small delay between readings
        if (i < NUM_READINGS - 1) { delay(READING_DELAY); }
    }

    // Find the color with the highest count

```

```

    int maxCount = 0; DetectedColor dominantColor = COLOR_UNDEFINED;
    for (int i = 0; i < 4; i++) { if (colorCounts[i] > maxCount) { maxCount = colorCounts[i];
    dominantColor = (DetectedColor)i; } }

    // Print results
    Serial.print("Color counts: RED="); Serial.print(colorCounts[COLOR_RED]);
    Serial.print(", WHITE="); Serial.print(colorCounts[COLOR_WHITE]); Serial.print(",
    BLACK="); Serial.print(colorCounts[COLOR_BLACK]); Serial.print(",
    UNDEFINED="); Serial.println(colorCounts[COLOR_UNDEFINED]);
    Serial.print("Dominant color: "); switch (dominantColor) { case COLOR_RED:
    Serial.println("RED"); break; case COLOR_WHITE: Serial.println("WHITE"); break;
    case COLOR_BLACK: Serial.println("BLACK"); break; case COLOR_UNDEFINED:
    Serial.println("UNDEFINED"); break; }

    // Set current color to the dominant color
    currentColor = dominantColor;

    // Reset stabilization flag
    waitingForStabilization = false;

    // Move to detected color state
    currentState = DETECTED_COLOR;
    stateTimer = millis();
}
} else { waitingForStabilization = false; }
}

```